

Outreach Engine

Handover and Knowledge Transfer Document

Field	Value
Product	Outreach Engine (repository <code>mailapp</code>)
Document version	1.2
Date	2026-09-24
Code baseline	commit <code>66a8539</code> on branch <code>main</code>
Production URL	https://outreach.somvanshitechnologies.digital
Audience	Engineers and operators taking over build, deployment and day-to-day operation

This document is the single entry point for anyone inheriting the platform. It explains what the system does, how it is built, how it runs in production, how to operate it, and what is unfinished or fragile. The existing repository documents remain the detailed references and are linked from each section:

- `README.md` – quick start and command reference
- `PLAN.md` – original architecture and delivery plan
- `docs/API.md` – HTTP contract (partially out of date, see section 12)
- `docs/AWS_SETUP.md` – AWS deployment guide
- `docs/OPERATIONS.md` – runbook
- `docs/TESTING.md` – test suites
- `infra/iam/README.md` – IAM policy for running locally against AWS

Table of contents

1. What the product does
2. Glossary
3. System architecture
4. Technology stack
5. Repository layout
6. Environments
7. Configuration
8. Data model
9. End-to-end flows
10. Access control and security
11. Background jobs and schedules
12. HTTP API
13. Dashboard (web application)
14. AWS infrastructure
15. Deployment procedure
16. Operations runbook
17. Testing and quality
18. Known issues, gaps and technical debt
19. Accounts, credentials and third parties
20. Handover checklist
21. Appendix A: environment variables
22. Appendix B: converting this document to PDF

1. What the product does

Outreach Engine is an internal B2B cold-outreach platform. An operator uploads a spreadsheet of leads; the system researches every lead, drafts a personalised email that pitches the best-fitting services from a catalogue, sends it through Amazon SES, tracks every delivery event, runs a follow-up sequence, and reports everything on a dashboard. A status spreadsheet can be exported at any time.

The pipeline for one lead:

```
Excel row
|
v
Research --> persona JSON (company, role, priorities, pain points, signals)
|           website fetch + Claude with server-side web search
v
Draft     --> subject + body + matched services + rationale
|           Claude, structured output, prompt-cached instruction prefix
v
Validate  --> deterministic checks (length, banned phrases, links, placeholders,
|           do-not-contact domains). One automatic retry with feedback.
v
Review    --> manual queue (approve / edit / regenerate / reject) or auto-approve
|
v
Send      --> SES v2 SendEmail inside the send window, daily cap and rate limit
|
v
Events    --> SES -> SNS -> webhook: Delivery, Open, Click, Bounce, Complaint ...
|
v
Follow-up --> next sequence step after N days unless replied / bounced /
|           complained / unsubscribed
```

Key product decisions (agreed with the product owner on 2026-09-23):

- **Delivery mode** `personal` (default) makes mail look hand-written: no `List-Unsubscribe` headers, a plain opt-out sentence, text-only unless open tracking is on. `bulk` is the classic newsletter shape. This controls Gmail Primary vs Promotions placement.
- **Sender identity precedence**: campaign override, then campaign owner's profile, then organisation settings.
- **Instruction documents**: a user's personal documents override the organisation's per kind for campaigns they own. Organisation documents are admin-only.
- **Campaign access**: admins see everything; the creator has full access; anyone else needs an explicit grant (view, edit or full). Ungranted campaigns are invisible (404), not forbidden.
- **Campaign settings lock** once the campaign leaves `draft`.
- **Admin kill switch** "stop sending" pauses every active campaign a user owns.

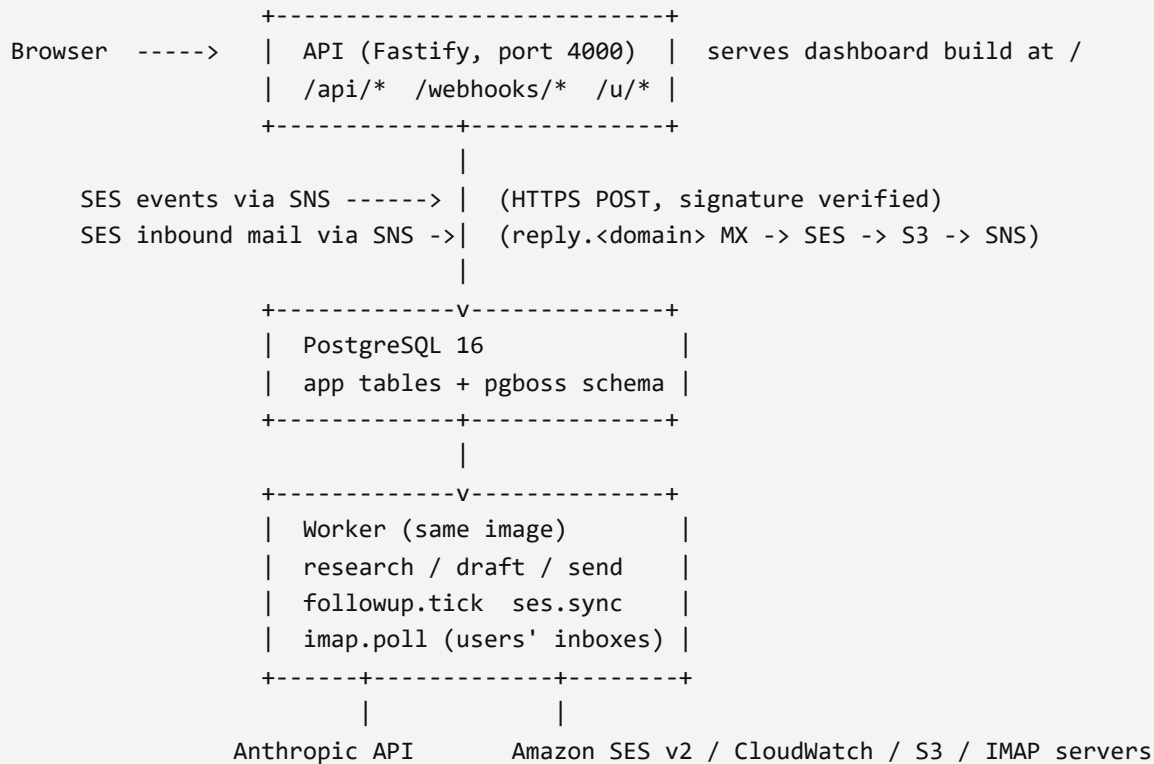
- **Reply capture, both ways** (decided 2026-09-23): replies go to the SES inbound domain by default and are forwarded to the campaign owner's real mailbox; users who insist on their own Reply-To have their mailbox polled over IMAP instead.
- **Link page instead of a bare unsubscribe link** (2026-09-24): the footer of every email carries one small "Manage preferences" link to a page that presents the service catalogue with link and contact buttons. The unsubscribe button on that page is **off by default** because outreach goes to known contacts who opt out by replying; admins can switch it on (with a "Do you really want to leave us?" confirmation) and unsubscribed contacts can subscribe again from the same page. One-click unsubscribe from mail clients (RFC 8058 POST) keeps working regardless.

2. Glossary

Term	Meaning
Lead	One row of the uploaded sheet: a person at a company with an email address
Campaign	One uploaded sheet plus its sequence, services, approval mode and sender overrides
Sequence / step	Ordered list of emails per lead (step 1 initial, steps 2..n follow-ups) with day delays
Persona	Structured research output stored on the lead
Service	An offering in the catalogue the model can pitch
Instruction doc	Text guiding the model: company_profile, tone, format, rules, signature, followup_guidance, other
Hard rules	Deterministic constraints enforced by the validator and the sender (word limits, banned phrases, send window, do-not-contact domains)
Delivery mode	<code>personal</code> or <code>bulk</code> ; shapes headers, HTML and footer of outbound mail
Suppression	An address that must never be emailed (hard bounce, complaint, unsubscribe, manual, SES account list)
Configuration set	SES resource that routes sending events to SNS
pg-boss	Postgres-backed job queue used by the worker (no Redis)
SES	Amazon Simple Email Service (v2 API)
SNS	Amazon Simple Notification Service; delivers SES events to the webhook
Org doc / personal doc	Instruction doc with <code>owner_id</code> null (organisation) or set (a user's own)
Inbound domain	<code>reply.somvanshitechnologies.digital</code> : SES receives mail for it and posts it to the app; generated Reply-To addresses live on it
Link page	The public page behind the "Manage preferences" link in every email (<code>/u/<token></code>): service catalogue, optional unsubscribe, subscribe again
IMAP cursor	Per-mailbox record of the last processed UID so polling is read-only and never repeats a message

3. System architecture

3.1 Components



Three processes run from one Docker image:

Process	Entry point	Responsibility
API	<code>node apps/api/dist/server.js</code>	HTTP API, dashboard static files, SNS webhooks, unsubscribe pages, publishes jobs
Worker	<code>node apps/api/dist/worker.js</code>	Runs all queued jobs and cron ticks
Migrate	<code>node apps/api/dist/db/migrate.js</code>	Applies SQL migrations from <code>apps/api/drizzle</code>

The API never does LLM or SES work inline (except the "send test" endpoint). Everything slow goes through pg-boss and the worker.

3.2 Request lifecycle (API)

- Fastify plugins: sensible, helmet (CSP in production), CORS (credentials), cookie, global rate limit (600 requests per minute per IP, webhooks and health probes exempt), multipart (25 MB, one file), auth plugin.
- `text/plain` bodies are parsed as JSON (SNS posts with that content type). `application/x-www-form-urlencoded` is parsed for RFC 8058 one-click unsubscribe.

3. Every response carries `x-request-id`. Errors are normalised to `{ error: { code, message, details?, requestId } }`.
4. Routes are registered per module (`apps/api/src/modules/*/routes.ts`).
5. If `apps/web/dist/index.html` exists the API serves it at `/` with an SPA fallback.

3.3 Application context

`apps/api/src/bootstrap.ts` builds one `AppContext` per process: config, Drizzle db handle, pino logger, metrics sink (CloudWatch or no-op), job queue (pg-boss), storage (local or S3), settings service, auth service, audit service, LLM provider (Anthropic or mock), SES gateway (SES v2 or mock). All route handlers and job handlers receive this context.

4. Technology stack

Layer	Technology	Version (package.json)
Language	TypeScript, Node.js	TS 5.6, Node >= 20 (dev machine has 25.2)
API	Fastify	5.1
ORM / migrations	Drizzle ORM, drizzle-kit	0.38 / 0.30
Database	PostgreSQL	16 (RDS and docker image)
Job queue	pg-boss	10.1
Logging	pino	9.5
Validation	zod	3.24
Auth	jose (JWT), @node-rs/argon2	5.9 / 2.0
LLM	@anthropic-ai/sdk	0.80
Email	@aws-sdk/client-sesv2, sns-validator, mailparser, imapflow	3.700 / 0.3.5 / 3.7 / 1.0
Spreadsheets	exceljs	4.4
HTML scraping	cheerio	1.0
Dashboard	React 18, Vite 5, React Router 6, TanStack Query 5, Tailwind 3, Recharts 2	see apps/web/package.json
Tests	Vitest 2, Testing Library, aws-sdk-client-mock	
Infrastructure	AWS CDK v2 (aws-cdk-lib 2.170)	
Container	Docker multi-stage image, public.ecr.aws/docker/library/node:20-bookworm-slim, tini, non-root user	

Default Claude model everywhere is `claude-opus-5` (env `LLM_MODEL`, `LLM_RESEARCH_MODEL`, and the database settings `llmModel`, `researchModel`).

5. Repository layout

5.1 Workspaces

The repo is an npm workspace monorepo (`package.json` at the root).

Path	Package	Purpose
<code>packages/shared</code>	<code>@mailapp/shared</code>	Enums, zod schemas, DTO types, Excel header aliases, LLM output schemas. Shared by API and web
<code>apps/api</code>	<code>@mailapp/api</code>	Fastify API and worker, Drizzle schema and migrations, all integrations
<code>apps/web</code>	<code>@mailapp/web</code>	React dashboard
<code>infra/cdk</code>	<code>@mailapp/infra</code>	CDK app with five stacks
<code>infra/iam</code>		IAM policy JSON for local runs against AWS

5.2 API source map

```
apps/api/src
  server.ts / worker.ts    entry points
  bootstrap.ts, context.ts AppContext construction
  app.ts                  Fastify app, plugins, error handler, static dashboard
  config.ts               env validation (zod), production guards
  db/schema.ts            Drizzle tables and enums
  db/migrate.ts, seed.ts  migration runner, admin + default instruction seed
  jobs/queue.ts           PgBossQueue and MemoryQueue (tests)
  jobs/register.ts        job handlers, concurrency, cron schedules
  lib/                   crypto (tokens), errors, time (send window), upload, validate
  modules/auth            login, JWT, refresh tokens, users, RBAC, campaign access
  modules/settings        org settings (single JSON row), sender resolution
  modules/services        service catalogue + xlsx import
  modules/instructions    instruction docs (org / personal), upload, import
  modules/campaigns       campaign lifecycle, leads, access grants, export
  modules/emails          review queue: approve / reject / regenerate / send-test
  modules/pipeline        research, draft, validator, render, website fetch
  modules/llm             Anthropic provider, mock provider, prompts
  modules/ses             SES v2 gateway, mock, sender job, rate limiter,
                          events, inbound, IMAP, SNS verification, sync
  modules/followups       follow-up scheduler tick
  modules/suppressions    suppression list
  modules/analytics       overview, timeseries, events, SES metrics, LLM usage
  modules/system          status, audit log
  modules/unsubscribe     /u/:token link page (landing.ts renders it), unsubscribe,
resubscribe
  modules/storage         local / S3 upload storage
  observability           pino logger, CloudWatch metrics sink
```

5.3 Storage module

`apps/api/src/modules/storage/storage.ts` implements `Storage` (`put`, `get`, `getFromBucket`) for a local folder (`STORAGE_LOCAL_DIR`, buckets emulated as sub-folders of `_buckets/`) and for S3 (`STORAGE_S3_BUCKET` + prefix). `getFromBucket` is what reads raw inbound mail from the SES inbound bucket. The `.gitignore` rule is anchored to `/storage/` so this module is tracked.

5.4 Web source map

<code>apps/web/src</code>	
<code>App.tsx</code>	routes (see section 13)
<code>components/</code>	<code>Layout</code> (<code>nav</code>), <code>ProtectedRoute</code> , <code>ReviewQueue</code> , <code>SequenceEditor</code> , <code>HeaderMappingPreview</code> , <code>ChipsInput</code> , <code>StatTile</code> , <code>StatusBadge</code> , <code>charts</code> , <code>ui</code>
<code>hooks/useAuth.tsx</code>	session state, login/logout, refresh on 401
<code>hooks/useToast.tsx</code>	notifications
<code>lib/api.ts</code>	fetch wrapper: CSRF header, refresh-on-401, error parsing
<code>pages/</code>	one file per screen (<code>LandingPagePage</code> = "Manage link page", <code>ProfilePage</code> = sender identity + IMAP mailbox + password)

5.4a Deploy helper

`infra/cdk/deploy-watchdog.ps1` wraps `cdk deploy` with a stall watchdog (section 15.2).

5.5 Root files

File	Purpose
<code>Dockerfile</code>	Multi-stage production image (deps, build, prod-deps, runtime)
<code>docker-compose.yml</code>	Local Postgres (port 5433), API and worker from the image
<code>.env.example</code>	Documented defaults for every environment variable
<code>tsconfig.base.json</code>	Shared TypeScript config
<code>PLAN.md</code> , <code>README.md</code> , <code>docs/</code>	Documentation

6. Environments

6.1 Local development

Everything runs on the developer machine with mocks; nothing leaves the machine.

```
npm install
cp .env.example .env           # LLM_PROVIDER=mock, SES_MODE=mock, Postgres on 5433
docker compose up -d postgres
npm run db:migrate
npm run db:seed                 # admin@example.com / ChangeMe!12345 + default docs
npm run dev:api                 # http://localhost:4000
npm run dev:worker              # second terminal
npm run dev:web                 # http://localhost:5173, proxies /api, /u, /webhooks
```

Mock behaviours (useful for demos and tests):

- Mock LLM returns a deterministic persona and draft from the row. The `notes` column can carry `mock:fail-research`, `mock:fail-draft`, `mock:long-draft`, `mock:banned` to force failures.
- Mock SES returns fake message ids; recipients starting with `fail@` raise `MessageRejected`.
- The mock SES gateway treats the configured from-address and its domain as verified identities.

To run locally against real AWS, follow `infra/iam/README.md` (policy `mailapp-dev-policy.json`), set `SES_MODE=ses`, `AWS_REGION`, credentials or `AWS_PROFILE`, and `SES_CONFIGURATION_SET`. Events only arrive if `PUBLIC_BASE_URL` is publicly reachable (for example through an ngrok tunnel) and an SNS subscription points at it.

6.2 Production

Item	Value
AWS account	176032258686
Region	ap-south-1 (Mumbai)
Public URL	https://outreach.somvanshitechnologies.digital
CDK stacks	MailApp-prod-Network, -Data, -Messaging, -Compute, -Observability
ECS cluster	mailapp-prod (services mailapp-prod-api, mailapp-prod-worker; task family mailapp-prod-migrate)
Database	RDS PostgreSQL 16, db.t4g.micro, single AZ, 20 GB gp3 (auto-grows to 100 GB)
SES configuration set	mailapp-prod-events
SES identity	Domain somvanshitechnologies.digital, verified manually in the SES console (not CDK managed)

Item	Value
Default from address	vigneyabhatt@somvanshitechnologies.digital ("Vigneya Bhatt")
SNS topics	mailapp-prod-ses-events and mailapp-prod-ses-inbound (both subscribed to the API webhooks, confirmed), mailapp-prod-alerts (no subscribers)
Inbound domain	reply.somvanshitechnologies.digital, MX at the registrar to inbound-smtp.ap-south-1.amazonaws.com; receipt rule set mailapp-prod-inbound (active) stores mail in the inbound bucket
TLS certificate	ACM arn:aws:acm:ap-south-1:176032258686:certificate/11374930-18d1-4f92-a6a7-90bcd7286255
DNS	Managed at the registrar, not Route 53 (hostedZoneId empty)
Task counts	1 API task, 1 worker task, no autoscaling
Caps	SES_DAILY_CAP=2000, SES_MAX_SEND_RATE=5 (defaults; editable in Settings)
Deploy credentials	IAM user mailer configured in the local AWS CLI
Inbound replies	Enabled: SES inbound domain + per-user IMAP polling (section 9.8)

All values above are in `infra/cdk/cdk.json` except the IAM user name and the registrar, which come from the previous owner's notes.

7. Configuration

Configuration has two layers.

7.1 Environment variables (process level)

Validated by `apps/api/src/config.ts` with `zod` at start-up. Invalid or missing values stop the process with a list of problems. Production guards: `ANTHROPIC_API_KEY` is required when `LLM_PROVIDER=anthropic`; `JWT_SECRET` and `APP_SECRET` must not start with `change-me`; SNS signature verification is forced on. The full variable list is in Appendix A.

In production the variables come from the ECS task definition (`infra/cdk/lib/compute-stack.ts`) and the secrets from Secrets Manager:

Secret name	Content
<code>mailapp/prod/db-credentials</code>	RDS master user/password (generated by CDK)
<code>mailapp/prod/DATABASE_URL</code>	Full <code>postgres://</code> URL with <code>sslmode=no-verify</code>
<code>mailapp/prod/JWT_SECRET</code>	64-char generated
<code>mailapp/prod/APP_SECRET</code>	64-char generated (HMAC for unsubscribe tokens)
<code>mailapp/prod/external</code>	JSON <code>{ "ANTHROPIC_API_KEY": "...", "IMAP_PASSWORD": "" }</code> filled by the operator. <code>IMAP_PASSWORD</code> belongs to the optional organisation-wide mailbox only; users' own mailbox passwords are stored encrypted in the database (<code>users.imap_password_enc</code> , AES-256-GCM keyed from <code>APP_SECRET</code>)

7.2 Organisation settings (database, editable in the dashboard)

Stored as one JSON row in table `settings` (key `app`), seeded from env defaults, cached in memory for 10 seconds. Admins edit them under **Settings**. Fields:

Field	Default	Notes
<code>fromEmail</code> , <code>fromName</code> , <code>replyTo</code>	from env	Organisation-wide sender; must be a verified SES identity (checked on save)
<code>postalAddress</code>	empty	Printed in the footer
<code>configurationSet</code>	from env	SES configuration set name
<code>dailyCap</code>	env (2000 in prod)	Emails per UTC day across all campaigns. <code>0</code> means unlimited, not paused
<code>maxSendRate</code>	env (5)	Per second; never exceeds the SES account rate
<code>defaultApprovalMode</code>	<code>manual</code>	<code>manual</code> or <code>auto</code>

Field	Default	Notes
llmModel, researchModel	claude-opus-5	
webSearchEnabled	true	Combined with env LLM_WEB_SEARCH
trackOpens	true	In personal mode adds a bare HTML part so the SES pixel works
trackClicks	true	
deliveryMode	personal	personal or bulk
forwardRepliesToOwner	true	Replies received on the SES inbound domain are forwarded to the campaign owner's real mailbox with Reply-To set to the lead
landingPage	see below	Content of the link page (Services → Manage link page)
hardRules	see below	Global rules; a campaign may override any subset

Link page (LandingPageSchema), edited under **Services → Manage link page** with a live preview:

Field	Default	Notes
emailLinkLabel	Manage preferences	Text of the small link in the email footer (HTML part; plain text still shows the URL)
headline, intro, footerNote	generic	Page wording
services[]	empty = every active service	Ordered list of { serviceId, showLink, showContact, contactUrl }; the link button uses the service url, the contact button uses contactUrl or mailto:contactEmail
linkLabel, contactLabel, contactEmail	Learn more, Contact us, empty	Button labels and default contact target
showUnsubscribe	false	Small unsubscribe button with a "Do you really want to leave us?" confirmation; unsubscribeLabel, unsubscribeNote set its wording

Hard rules (HardRulesSchema in packages/shared/src/schemas.ts):

Rule	Default
maxWords / minWords	180 / 40
maxSubjectChars	80
bannedPhrases, requiredPhrases	empty
forbidLinks / maxLinks	false / 2
forbidEmojis, forbidAllCapsWords	true
forbidExclamation	false

Rule	Default
<code>requireUnsubscribeFooter</code>	true
<code>doNotContactDomains</code>	empty
<code>sendWindowStartHour</code> / <code>sendWindowEndHour</code>	8 / 18 (local hour in <code>timezone</code>)
<code>sendDays</code>	Monday to Friday (<code>[1,2,3,4,5]</code>)
<code>timezone</code>	UTC

7.3 Precedence rules

- Sender identity: campaign `fromEmail/fromName` > campaign owner's profile > settings.
- Reply-To: campaign `replyTo` > owner's profile `replyTo` > `<from local part>@SES_INBOUND_DOMAIN` (when the inbound domain is configured) > settings `replyTo`. In production this means a campaign by `priya@somvanshitechnologies.digital` carries Reply-To: `priya@reply.somvanshitechnologies.digital` unless Priya set her own Reply-To.
- Postal address: owner's profile > settings.
- Instruction docs: owner's active personal docs of a kind > organisation docs of that kind.
- Hard rules: settings `hardRules` merged with campaign `hardRulesOverride`.
- Daily cap: minimum of settings `dailyCap` and the SES 24-hour quota (when known).
- Send rate: minimum of settings `maxSendRate` and the SES max send rate.

8. Data model

Schema: `apps/api/src/db/schema.ts`. Migrations: `apps/api/drizzle/0000_*.sql` (initial), `0001_access_and_sender_profiles.sql` (2026-09-23 feature batch) and `0002_reply_capture.sql` (IMAP mailbox columns and cursors). pg-boss owns the separate `pgboss` schema.

8.1 Tables

Table	Purpose and notable columns
<code>users</code>	Login accounts. <code>role</code> (admin/operator/viewer), lockout fields, personal sender identity (<code>from_email</code> , <code>from_name</code> , <code>reply_to</code> , <code>postal_address</code>), <code>dashboard_scope</code> (own/all), personal IMAP mailbox (<code>imap_enabled</code> , <code>imap_host</code> , <code>imap_port</code> , <code>imap_user</code> , <code>imap_password_enc</code> , <code>imap_mailbox</code>)
<code>imap_cursors</code>	Polling position per mailbox: <code>account_key</code> (<code>env</code> or <code>user:<id></code>), <code>uid_validity</code> , <code>last_uid</code> , <code>last_polled_at</code> , <code>last_error</code>
<code>refresh_tokens</code>	Hashed rotating refresh tokens with revocation
<code>campaign_access</code>	Admin grants: (<code>campaign_id</code> , <code>user_id</code>) unique, <code>level</code> view/edit/full
<code>audit_logs</code>	Every mutating action: user, action, entity, metadata, ip
<code>settings</code>	Single JSON row (<code>key</code> = app)
<code>services</code>	Catalogue: name, description, target audience, value props, proof points, url, tags, <code>is_active</code> (soft delete)
<code>instruction_docs</code>	kind, title, content, <code>version</code> , <code>is_active</code> , <code>owner_id</code> (null = organisation)
<code>files</code>	Uploaded sheets, instruction uploads, exports, raw inbound mail; <code>storage</code> local/s3 + <code>key</code>
<code>campaigns</code>	name, status, approval mode, <code>sequence</code> (JSON steps), <code>service_ids</code> , sender overrides, <code>extra_guidance</code> , <code>hard_rules_override</code> , source file, <code>header_map</code> , <code>import_summary</code> , <code>created_by</code> (owner)
<code>leads</code>	One per campaign row; unique (<code>campaign_id</code> , <code>email</code>); mapped columns + <code>extra</code> JSON; <code>status</code> , <code>current_step</code> , <code>next_action_at</code> ; <code>persona</code> , <code>research_raw</code> , <code>matched_services</code> ; <code>unsubscribe_token</code> (unique); event timestamps
<code>emails</code>	Outbound drafts/sent mail and inbound replies (<code>direction</code>). Step, status, subject, <code>body_text</code> , <code>body_html</code> , SES ids and RFC headers (<code>ses_message_id</code> , <code>message_id_header</code> , <code>in_reply_to</code> , <code>references_header</code>), <code>llm_meta</code> , <code>validation</code> , review fields, <code>scheduled_for</code> , <code>sender_user_id</code> , <code>raw_send_response</code>
<code>send_attempts</code>	One row per SES SendEmail attempt: request summary, response or error, duration
<code>email_events</code>	Every SES event; unique <code>dedupe_key</code> = <code>type:messageId:occurredAt</code>
<code>suppressions</code>	Unique email + reason (<code>hard_bounce</code> , <code>complaint</code> , <code>unsubscribe</code> , <code>manual</code> , <code>ses_account_list</code>)
<code>inbound_messages</code>	Parsed replies with match method and auto-reply flag; unique (<code>source</code> , <code>external_id</code>)

Table	Purpose and notable columns
ses_snapshots	Periodic account, metrics, suppression_sync snapshots
daily_send_counters	day (UTC) to count; atomic daily cap reservation
llm_calls	Token usage, latency and outcome of every model call

8.2 Lead status machine

Statuses: pending, researching, researched, drafting, pending_review, approved, scheduled, sending, sent, delivered, opened, clicked, replied, bounced, complained, unsubscribed, suppressed, invalid, rejected, failed, completed, skipped.

```

pending -> researching -> researched -> drafting -> pending_review -> approved
                                     \-> approved (auto mode)
approved -> scheduled (outside window / cap) -> sending -> sent
sent -> delivered -> opened -> clicked          (upgrades only, never downgrades)
sent|delivered|opened|clicked -> (follow-up due) -> drafting ... next step
any non-terminal -> replied | bounced | complained | unsubscribed | suppressed
                  | rejected | failed | skipped | completed

```

Terminal (no further work): replied, bounced, complained, unsubscribed, suppressed, invalid, rejected, completed, skipped. **failed** is not terminal: **Retry** re-queues it. Follow-up eligible: sent, delivered, opened, clicked.

8.3 Email status machine

```

draft -> pending_review -> approved -> queued -> sending -> sent
                                                -> delivered
                                                -> bounced | complained
approved/queued -> rejected (lead replied, suppressed, reviewer)
sending -> failed (permanent SES error) or back to approved (transient, retried)

```

8.4 Campaign status machine

```

draft -> active (start)      active -> paused (pause)      paused -> active (resume)
active -> completed (automatic when no lead can progress)
draft | paused | completed | active -> archived
delete: only draft or archived, admin only
PATCH settings: only while draft

```

9. End-to-end flows

9.1 Import

POST /api/campaigns/preview then POST /api/campaigns (multipart: file + JSON payload).

1. `parseSheet` reads xlsx or csv with `exceljs`.
2. `mapHeaders` maps headers to canonical keys case-insensitively via aliases (`packages/shared/src/excel.ts`). Only `email` is required. Recognised: first name, last name, company, website, job title, linkedin, industry, location, phone, notes. Unknown columns go to `extra` and come back in the export. Gotcha: a column literally named `name` maps to `first_name`.
3. Rows are validated (email syntax), de-duplicated within the sheet, and checked against `suppressions`; suppressed rows are created with status `suppressed`.
4. The file is stored (local or S3) and referenced from `files`; the campaign gets `header_map`, `original_headers`, `import_summary`.
5. Leads are inserted in chunks of 500; each gets an HMAC unsubscribe token derived from its id.

9.2 Start and enqueue

POST /api/campaigns/:id/start (needs full access) sets `active` and calls `enqueuePendingWork`: pending/researching leads get `lead.research`; researched/drafting get `lead.draft`; approved/scheduled get `email.send` for their latest approved email. Jobs are singleton-keyed per lead and stage, so start and resume are safe to repeat.

9.3 Research (`lead.research`)

`apps/api/src/modules/pipeline/research.ts`

1. Skip if the lead is terminal or the campaign is not active.
2. Normalise the website URL (falls back to the email domain) and fetch an extract (`WEBSITE_FETCH_ENABLED`).
3. Load the instruction bundle for the campaign owner (company profile) and the selected services.
4. Anthropic provider: phase 1 is an agentic loop (up to 6 iterations) with server-side `web_search` (max 6 uses) and `web_fetch` (max 4 uses), adaptive thinking, medium effort. Phase 2 converts the findings into `PersonaSchema` with structured outputs.
5. Persona and raw findings are stored on the lead (`researched`) and `lead.draft` step 1 is published. Failures set `failed` with `last_error = research: ...` and rethrow so pg-boss retries.

9.4 Draft and validate (`lead.draft`)

`apps/api/src/modules/pipeline/draft.ts`

1. Guards: lead not terminal, has persona, campaign active, step exists in the sequence, no existing non-rejected draft for that step (unless regenerating).

2. Resolve sender, load instruction bundle for the sender owner, active services filtered by `campaign.service_ids`, previous emails in the thread (sent or inbound).
3. Call the model with a system prompt built from company profile, catalogue, tone, format, rules and hard rules. The last system block carries `cache_control: ephemeral` so the stable prefix is prompt-cached across all leads of a campaign. Output is `DraftOutputSchema` (subject, body, selected services with fit score and rationale, pitch angle, call to action, self-check).
4. Run `validateDraft`. On error the model gets one more attempt with the validator's feedback.
5. Store the email as `pending_review` (manual mode, or validation still failing) or `approved` (auto mode and valid). Follow-up steps with `threaded=true` get `Re: <original subject>`. Approved emails publish `email.send` immediately.

Validator rules (errors block auto-approval; warnings are shown to reviewers): empty subject, subject length, word count min/max, banned and required phrases, link count or links forbidden, emoji, exclamation marks, ALL-CAPS words, unfilled placeholders like `[Name]` or `{{company}}`, recipient domain on the do-not-contact list. Warnings: fake "Re:", unsubscribe wording in the body, generic greeting, cliché opener.

9.5 Review

Dashboard **Review queue** (GET `/api/emails?status=pending_review`). Actions per email:

Action	Endpoint	Effect
Approve (optionally edited)	POST <code>/api/emails/:id/approve</code>	Re-validates; reviewers may override validator errors except do-not-contact domains; queues send
Reject	POST <code>/api/emails/:id/reject</code>	Email and lead become <code>rejected</code>
Regenerate (with feedback)	POST <code>/api/emails/:id/regenerate</code>	Current email rejected; new draft job with feedback
Send test	POST <code>/api/emails/:id/send-test</code>	Sends the rendered email to an address you own with subject prefix <code>[TEST]</code>
Approve all	POST <code>/api/campaigns/:id/approve-all</code>	Approves every pending draft that passed validation

Approve, reject and regenerate need `edit` access to the campaign.

9.6 Send (`email.send`)

`apps/api/src/modules/ses/sender.ts`, in this order:

1. Email must be approved/queued/sending; lead not terminal (else the email is rejected); campaign active (else the email stays approved and the job ends).
2. Suppression re-check at send time.
3. Send window: if outside `sendDays` and hours in `timezone`, set `queued` with `scheduled_for` and re-publish with `startAfter`. The lead shows `scheduled` and the dashboard shows the expected send time.

4. Daily cap: atomic increment of `daily_send_counters` bounded by `min(settings cap, SES quota)`. If exhausted, reschedule to 00:05 UTC next day and emit `send_rate_limited{reason=daily_cap}`.
5. Per-second token bucket (`TokenBucket`) at `min(settings rate, SES rate)`. The bucket is per worker process.
6. Render text and HTML via `renderEmail` according to delivery mode, signature (instruction doc), opt-out sentence, postal address. The HTML part carries one small link (`landingPage.emailLinkLabel` , default "Manage preferences") to `/u/:token`; the plain-text part spells the URL out. In personal mode the HTML part only exists when `trackOpens` is on.
7. Threading for steps > 1: `In-Reply-To` and `References` from the last sent email.
8. SES v2 `SendEmail` with configuration set, tags (`campaign_id`, `lead_id`, `email_id`, `step`) and extra headers (`List-Unsubscribe` only in bulk mode).
9. On success: `send_attempts` row, email sent with `ses_message_id`, `message_id_header` (`<id@ap-south-1.amazonses.com>`), `sender_user_id`; lead sent, `current_step`, and `next_action_at = now + next step delayDays`.
10. On error: permanent (`MessageRejected`, `MailFromDomainNotVerifiedException`, `AccountSuspendedException`, `BadRequestException`, `NotFoundException`, `SendingPausedException`, or any 4xx except 429) marks email and lead `failed`; transient errors keep `approved` and rethrow so pg-boss retries (3 attempts, 30 s backoff).

9.7 Events (`POST /webhooks/ses/events`)

1. `verifySnsEnvelope` validates the SNS signature (always in production) and, if `SNS_ALLOWED_TOPIC_ARNS` is set, the topic.
2. `SubscriptionConfirmation` is confirmed automatically by fetching `SubscribeURL`.
3. `processSesEvent` finds the email by `ses_message_id` or `message_id_header`, stores the event idempotently (dedupe key), and applies it:

Event	Effect
Send	stored only
Delivery	email <code>delivered</code> ; lead <code>delivered_at</code> , status upgraded if lower
Open, Click	lead <code>opened_at</code> / <code>clicked_at</code> , status upgraded. Clicks on <code>/u/</code> links are ignored
Bounce (Permanent)	email <code>bounced</code> ; lead <code>bounced</code> , sequence stops; suppression added locally and in the SES account list
Bounce (Transient)	lead <code>last_error</code> soft bounce note only
Complaint	email and lead <code>complained</code> ; suppression local and SES
Reject, RenderingFailure	email and lead <code>failed</code>
DeliveryDelay	lead <code>last_error</code> note
Subscription	lead <code>unsubscribed</code> ; suppression

Events for unknown message ids are stored with null references (useful for send-test mails).

9.8 Replies

Both mechanisms are live in production and feed `recordInbound`:

- **SES inbound (primary).** Emails whose owner has no personal Reply-To carry `Reply-To: <sender>@reply.somvanshitechnologies.digital`. The MX of that sub-domain points at SES; the receipt rule set `mailapp-prod-inbound` stores the raw MIME in the inbound bucket and notifies `mailapp-prod-ses-inbound`, which posts to `POST /webhooks/ses/inbound`. The API downloads the object, stores a `files` row (`inbound_raw`) and parses it with `mailparser`. Spam or virus verdict `FAIL` is dropped. Because nobody reads that mailbox, a matched reply is **forwarded** to the campaign owner's real mailbox (`forwardRepliesToOwner`, on by default): subject `Re: ...`, Reply-To set to the lead, the lead page link and the quoted reply in the body, sent from the organisation from-address.
- **IMAP polling (fallback).** Every 2 minutes the worker polls the optional organisation mailbox (`IMAP_*` env) and every user whose profile has **Reply polling** enabled (`users.imap_*`; password decrypted with `APP_SECRET`). Polling is read-only: it keeps a UID cursor per mailbox in `imap_cursors`, only reads messages above the cursor (the last 3 days on first contact or after a `UIDVALIDITY` change), and never changes flags. Errors are recorded per mailbox and shown on My profile and the System page. IMAP replies are not forwarded, since they already sit in the owner's inbox.

Matching: In-Reply-To and References against `ses_message_id / message_id_header`; fallback is the sender address matching a lead emailed in the last 90 days. Auto-replies (headers `Auto-Submitted`, `X-Autoreply`, `Precedence: bulk`, out-of-office subjects) are stored but do not change the lead. A real reply sets `replied`, cancels pending drafts and approved follow-ups, and adds an inbound `emails` row shown on the lead page. Manual fallback: `POST /api/leads/:id/mark-replied`.

9.9 Follow-ups (`followup.tick` , every minute)

For up to 500 leads in follow-up-eligible statuses whose `next_action_at <= now` in active campaigns: if a next step exists, clear `next_action_at` and publish `lead.draft` for that step; otherwise mark `completed`. Then sweep active campaigns and mark `completed` those where no lead can progress (every lead terminal, failed, or sent the last step).

9.10 Link page, unsubscribe and subscribe again

`GET /u/:token` renders the link page (`modules/unsubscribe/landing.ts`) from `settings.landingPage` and the active service catalogue: headline and intro, one card per selected service (description, up to three value props, "Learn more" to the service URL, "Contact us" to `contactUrl` or `mailto:contactEmail`), a footer note and, only when `showUnsubscribe` is on, a small unsubscribe button. That button opens an inline confirmation ("Do you really want to leave us?" with "Yes, unsubscribe" / "No, stay"); only Yes submits.

- `POST /u/:token` (the confirmation form, or an RFC 8058 one-click POST from a mail client, which works even when the button is hidden): lead `unsubscribed`, suppression `unsubscribe`, pending emails cancelled, audit entry. The page then shows the unsubscribed notice with a **Subscribe again** button.
- `POST /u/:token/resubscribe`: removes the `unsubscribe` suppression (bounce and complaint suppressions are never lifted), clears `unsubscribed_at` on every lead with that address and closes

their sequence (`completed` if they had been emailed, else `skipped`); future campaigns may email them again. Metric `resubscribes`, audit `lead.resubscribe_link`.

- Invalid tokens render the same page with a "link not valid" notice (HTTP 404).

Tokens are HMAC(`APP_SECRET`, `leadId`); rotating `APP_SECRET` invalidates every link already sent. `POST /api/settings/landing/preview` renders a draft configuration for the editor's preview pane.

9.11 Exports

Export	Endpoint	Content
Campaign status	<code>GET /api/campaigns/:id/export</code>	Original columns + <code>status</code> , <code>current_step</code> , <code>matched_services</code> , <code>subject</code> , <code>ses_message_id</code> , event timestamps, <code>last_error</code> , <code>last_updated_at</code>
Settings bundle	<code>GET /api/settings/export</code>	Settings, hard rules, organisation and own instruction docs, services (sheet "Instructions" is re-importable)
Suppressions	<code>GET /api/suppressions/export</code>	Full suppression list

9.12 SES sync (`ses.sync`, every 5 minutes and once at boot)

Stores `GetAccount` (quota, send rate, enforcement status, VDM) and CloudWatch `AWS/SES` series as snapshots, reconciles the SES account suppression list into `suppressions` (reason `ses_account_list`), and emits `queue_depth` metrics. The stored quota feeds the daily cap and rate limit in the sender.

10. Access control and security

10.1 Authentication

- Passwords: `argon2id` (`@node-rs/argon2`). A dummy hash is verified for unknown emails to equalise timing.
- Login: `POST /api/auth/login`, rate limited to 10 per minute per IP. Five failed attempts lock the account for 15 minutes.
- Session: JWT access token (15 min) in cookie `mailapp_access` (`httpOnly`, `SameSite=Lax`, path `/`, secure in production). Refresh token (14 days) in cookie `mailapp_refresh` (path `/api/auth`), stored hashed, rotated on every refresh, revoked on logout, password change, or deactivation.
- Authorization: Bearer `<jwt>` is also accepted (API clients).
- CSRF: cookie-authenticated non-GET requests must send `X-Requested-With: mailapp`.
- The dashboard calls `POST /api/auth/refresh` when it receives `401 token_expired`.

10.2 Roles

Role	Can
viewer	Read everything they can see; never above <code>view</code> on a campaign
operator	Create campaigns, review, manage services, personal instruction docs, suppressions, own profile
admin	Everything: users, organisation settings, organisation instruction docs, campaign access grants, delete campaigns/users, audit log

`requireRole(x)` accepts that role or higher (viewer < operator < admin).

10.3 Campaign-level access

Implemented in `apps/api/src/modules/auth/access.ts`; every campaign, lead and email endpoint goes through `requireCampaignAccess`, `requireLeadAccess` or `requireEmailAccess`.

Caller	Effective access
admin	full on every campaign
campaign creator	full (view if the creator is a viewer)
user with a grant	the granted level (view, edit, full), capped at view for viewers
anyone else	none: the campaign returns 404

Required levels: view for reading, leads, stats, export; edit for approve/reject/regenerate, send-test, approve-all; full for start/pause/resume/archive, settings PATCH, delete (admin only). List endpoints (`/api/campaigns`, `/api/emails`, analytics) are filtered by `visibleCampaignIds`. Analytics additionally honour `users.dashboard_scope = all`, which lets a non-admin see all-campaign numbers without campaign access.

10.4 Users administration

Endpoint	Behaviour
GET /api/users , GET /api/users/stats	List; per-user campaigns, active campaigns, sent total / 7 days / today, pending sends, last sent
GET /api/users/directory	id, name, email, role of active users (any role; used by the access-grant picker)
POST /api/users , PATCH /api/users/:id	Create / update incl. role, active flag, password reset, sender identity (verified against SES), dashboard scope. Cannot change own role or deactivate self
DELETE /api/users/:id	Cannot delete self or the last active admin. Pauses the user's active campaigns, keeps campaigns/emails/audit (owner set null), cascades personal docs, grants, tokens
POST /api/users/:id/stop-sending	Pauses every active campaign the user owns
PATCH /api/auth/me	Own name, sender identity and IMAP mailbox (imapEnabled, imapHost, imapPort, imapUser, imapPassword write-only, imapMailbox); enabling requires host, user and a stored password
POST /api/auth/me/imap-test	Connects to the mailbox with the submitted or stored credentials and reports the message count of the last 3 days

10.5 Other controls

- helmet with a strict CSP in production; CORS restricted to WEB_ORIGIN and PUBLIC_BASE_URL .
- Uploads: extension allow-lists per endpoint, 25 MB multipart limit (5 MB instruction upload, 10 MB instruction import), single file.
- SNS signature validation (sns-validator), optional topic allow-list.
- Unsubscribe tokens are HMAC-signed; unsubscribe routes are rate limited to 60 per minute.
- Audit log for every mutating action (audit_logs), visible to admins under **Audit log**.
- Secrets only from env / Secrets Manager; the ECS execution role can read only the five secrets.
- Task role is least privilege: SES send and account/suppression/identity reads, CloudWatch put (namespace MailApp) and read, S3 storage bucket read/write, inbound bucket read/delete.

11. Background jobs and schedules

Queue implementation: `apps/api/src/jobs/queue.ts` (pg-boss, schema `pgboss`, 5 connections, completed jobs archived after 7 days and deleted after 30). Every job defaults to 3 retries with 30 s exponential backoff and a 15 minute expiry. Handlers and schedules are in `apps/api/src/jobs/register.ts`.

Queue	Trigger	Concurrency	Handler
<code>lead.research</code>	campaign start/resume, retry, re-research	<code>WORKER_RESEARCH_CONCURRENCY</code> (2)	<code>runResearchJob</code>
<code>lead.draft</code>	after research, follow-up tick, regenerate	<code>WORKER_DRAFT_CONCURRENCY</code> (2)	<code>runDraftJob</code>
<code>email.send</code>	approve, auto-approve, reschedule	<code>WORKER_SEND_CONCURRENCY</code> (2)	<code>runSendJob</code>
<code>followup.tick</code>	cron <code>* * * * *</code> (UTC)	1	<code>runFollowupTick</code>
<code>ses.sync</code>	cron <code>*/5 * * * *</code> plus one job at worker boot	1	<code>runSesSync</code>
<code>metrics.flush</code>	cron <code>* * * * *</code>	1	flush buffered CloudWatch metrics
<code>imap.poll</code>	cron <code>*/2 * * * *</code> (always scheduled; polls the env mailbox if enabled plus every user mailbox with reply polling on)	1	<code>runImapPoll</code>
<code>campaign.export</code>	defined in <code>JOB_QUEUES</code> but unused; export is synchronous		

Concurrency is implemented as one pg-boss poller per slot (pg-boss 10 has no per-worker concurrency option). Scaling notes: `LLM_MAX_CONCURRENCY` x `worker tasks` must stay within the Anthropic rate limit; `WORKER_SEND_CONCURRENCY` x `worker tasks` should stay below the SES send rate because the token bucket is per process.

Queue health is visible on the dashboard **System** page (`/api/system/status`) and in `pgboss.job`.

12. HTTP API

`docs/API.md` documents conventions (auth cookies, CSRF header, error format, pagination) and the endpoints that existed at the initial commit. The 2026-09-23 feature batch added the endpoints below, which are **not yet in** `docs/API.md`:

Method and path	Role	Purpose
PATCH <code>/api/auth/me</code>	any	Update own name and sender identity
GET <code>/api/users/directory</code>	any	Active user picker
GET <code>/api/users/stats</code>	admin	Per-user sending statistics
DELETE <code>/api/users/:id</code>	admin	Delete user (see 10.4)
POST <code>/api/users/:id/stop-sending</code>	admin	Pause the user's active campaigns
GET <code>/api/campaigns/:id/access</code>	admin	List grants
PUT <code>/api/campaigns/:id/access</code>	admin	Upsert grant { <code>userId</code> , <code>level</code> }
DELETE <code>/api/campaigns/:id/access/:userId</code>	admin	Revoke grant
GET <code>/api/instructions?scope=org</code>	mine`	any
POST <code>/api/instructions</code> and <code>/upload</code> with field <code>scope</code>	operator (org scope admin only)	Create doc in a scope
POST <code>/api/instructions/import</code>	operator	Import docs from a workbook sheet "Instructions" (<code>kind</code> , <code>title</code> , <code>content</code>)
GET <code>/api/settings/export</code>	any	Settings workbook
GET <code>/api/suppressions/export</code>	any	Suppressions workbook
POST <code>/api/auth/me/imap-test</code>	any	Test own IMAP mailbox credentials
PUT <code>/api/settings/landing</code>	admin	Save the link page configuration (<code>LandingPageSchema</code>)
POST <code>/api/settings/landing/preview</code>	any	Render a draft link page as HTML
POST <code>/u/:token/resubscribe</code>	public	Subscribe an unsubscribed contact again

Behavioural changes to existing endpoints: PATCH `/api/campaigns/:id` returns 409 unless the campaign is a draft; GET `/api/campaigns` and GET `/api/emails` are filtered by visibility; campaign DTOs carry `createdByName` and `myAccess`; lead DTOs carry `nextSendAt`; POST `/api/emails/:id/*` require edit access; PUT `/api/settings` and user sender fields reject addresses that are not verified SES identities.

GET `/api/auth/me` and GET `/api/users` now include `imap` (status, never the password) and `effectiveReplyTo`; GET `/api/system/status` includes `replyCapture` (inbound domain and polled mailboxes with last poll time and error).

Public, unauthenticated routes: POST /webhooks/ses/events, POST /webhooks/ses/inbound, GET|POST /u/:token, POST /u/:token/resubscribe, GET /healthz, GET /readyz.

13. Dashboard (web application)

Single-page React app built by Vite into `apps/web/dist` and served by the API in production. Routes from `apps/web/src/App.tsx`; navigation from `components/Layout.tsx`.

Route	Page	Who	What it shows / does
<code>/login</code>	Login	public	Email + password
<code>/</code>	Dashboard	all	Outreach funnel and timeseries, SES account card (quota, enforcement), SES CloudWatch metrics, LLM usage, system status. Scoped by campaign visibility and dashboard scope
<code>/campaigns</code>	Campaigns	all	List with owner and access level; New campaign (operator+)
<code>/campaigns/new</code>	New campaign	operator, admin	Upload sheet, header mapping preview, sequence editor, service selection, approval mode, sender overrides, extra guidance, hard-rule overrides
<code>/campaigns/:id</code>	Campaign detail	by access	Counts, leads table with search/filter and expected send time, start/pause/resume/archive, approve-all, export, edit (draft only), access grants (admin)
<code>/leads/:id</code>	Lead detail	by access	Persona, matched services, email thread with events and send attempts, retry, re-research, skip, mark replied, unsubscribe
<code>/review</code>	Review queue	all (actions operator+)	Pending drafts across visible campaigns with validation issues; approve/edit/regenerate/reject/send test
<code>/services</code>	Services	all (edit operator+)	Catalogue CRUD and xlsx import; button to the link page editor
<code>/services/landing</code>	Manage link page	all (save admin)	Wording, contact email, per-service inclusion/order/buttons, unsubscribe button toggle, live preview of the public page
<code>/instructions</code>	Instructions	all	Organisation tab (admin edits) and My documents tab; upload md/txt/docx; import workbook; shows which kinds are personalised
<code>/suppressions</code>	Suppressions	all (edit operator+)	Search, add, remove, import, export
<code>/settings</code>	Settings	all (save admin)	Inbox placement (delivery mode), reply capture status and forwarding toggle, sender, caps, models, tracking, hard rules, postal address; export workbook
<code>/users</code>	Users	admin	Create/edit/delete users, roles, sender identity, dashboard scope, per-user stats, stop sending
<code>/audit</code>	Audit log	admin	Paginated audit entries
<code>/system</code>	System	all	Version, env, providers, DB/queue health, queue counts, last SES sync, sent today vs cap, reply capture (inbound domain, polled mailboxes)

Route	Page	Who	What it shows / does
/profile	My profile	all	Own name, sender identity (verified against SES) with the effective Reply-To, reply polling (IMAP host/user/password, test connection, last poll status), change password

Front-end conventions: TanStack Query for data, `lib/api.ts` adds the CSRF header and refreshes on 401 once, toasts for errors, Tailwind for styling, Recharts for charts.

14. AWS infrastructure

Defined in `infra/cdk` (TypeScript CDK v2). Context is read from `infra/cdk/cdk.json` (`infra/cdk/config.ts` documents every key). Stack names are `MailApp-<envName>-<Stack>`.

14.1 Stacks and resources

Stack	Resources
Network	VPC <code>10.42.0.0/16</code> , 2 AZs, no NAT gateway, subnet groups <code>public</code> and <code>app</code> (both public) and <code>data</code> (isolated), S3 gateway endpoint, security groups: ALB (80/443 from anywhere), app (4000 from ALB), db (5432 from app)
Data	RDS PostgreSQL 16 (encrypted, single AZ, 7-day backups, deletion protection and snapshot-on-delete in prod, <code>rds.force_ssl=1</code> , slow query log at 2 s, Performance Insights), Secrets Manager secrets (section 7.1), storage bucket (versioned, 90-day non-current expiry), inbound bucket (90-day expiry), log groups <code>/mailapp/prod/api</code> , <code>/worker</code> , <code>/migrate</code> (3 months retention in prod)
Messaging	SNS topics <code>mailapp-prod-ses-events</code> and <code>mailapp-prod-ses-inbound</code> ; SES configuration set <code>mailapp-prod-events</code> (reputation metrics, TLS required, account suppression for bounces and complaints) with an event destination for all 10 event types; optional SES domain identity with Easy DKIM and custom MAIL FROM (only when <code>sendingDomain</code> is set, which it is not in prod); inbound receipt rule set <code>mailapp-prod-inbound</code> for <code>inboundDomain</code> (drop spam, scan, store to the inbound bucket, notify the inbound topic) activated by an <code>AwsCustomResource</code> (<code>setActiveReceiptRuleSet</code>); the events webhook subscription
Compute	ECS cluster <code>mailapp-prod</code> with Container Insights; Docker image built from the repo root as a CDK asset (linux/amd64); task role and execution role; API Fargate service (0.5 vCPU, 1 GB) behind an internet-facing ALB with HTTPS (ACM cert) and HTTP redirect, health check <code>/healthz</code> , circuit breaker with rollback; worker Fargate service (0.5 vCPU, 1 GB, min healthy 0 so only one worker replaces another); migration task definition; the inbound webhook subscription (created here, after the API service rollout, because the API only confirms topics listed in <code>SNS_ALLOWED_TOPIC_ARNs</code>); outputs including <code>RunMigrationsCommand</code> and <code>InboundMxRecord</code>
Observability	Alerts SNS topic <code>mailapp-prod-alerts</code> (email subscription only if <code>alertEmail</code> set); alarms: SES bounce rate > 5 %, complaint rate > 0.1 %, ALB target 5xx > 10 per 5 min, unhealthy hosts, worker running count < 1, send failures > 5 per 5 min, LLM failures > 5 per 5 min, RDS free storage < 2 GiB, RDS CPU > 85 % for 15 min; CloudWatch dashboard <code>mailapp-prod</code>

Tasks get public IPs (no NAT) so they can reach ECR, Secrets Manager, SES, CloudWatch and the Anthropic API. Inbound traffic to tasks is limited to the ALB by security group.

14.2 Current production context (`infra/cdk/cdk.json`)

```
"envName": "prod",
"region": "ap-south-1",
"account": "176032258686",
"domainName": "outreach.somvanshitechnologies.digital",
"hostedZoneId": "",
```

```

"certificateArn": "arn:aws:acm:ap-south-1:176032258686:certificate/11374930-...",
"sendingDomain": "",
"fromEmail": "vigneyabhatt@somvanshitechnologies.digital",
"fromName": "Vigneya Bhatt",
"replyTo": "vigneyabhatt@somvanshitechnologies.digital",
"inboundEnabled": true,
"inboundDomain": "reply.somvanshitechnologies.digital",
>alertEmail": "",
"dbInstanceClass": "t4g.micro",
"apiDesiredCount": 1,
"workerDesiredCount": 1,
"imageUri": "",
"llmModel": "claude-opus-5",
"sesDailyCap": 2000,
"sesMaxSendRate": 5,
"subscribeWebhooks": true

```

14.3 SES state

- Domain identity `somvanshitechnologies.digital` is verified (DKIM) in the SES console, so any address on that domain can send. The application checks sender addresses against `ListEmailIdentities` when they are saved. Adding a sender on this domain needs no AWS work; a new domain must be verified first (procedure in 16.2a).
- SES production access must be active (otherwise only verified recipients and a 200/day quota).
- Virtual Deliverability Manager engagement tracking is enabled account-wide, which is what makes Open and Click events possible.
- The SNS topic subscription to `https://outreach.somvanshitechnologies.digital/webhooks/ses/events` must show **Confirmed**. On 2026-09-23 opens were visible in the SES console but not in the app because this subscription did not exist (`subscribeWebhooks` was false). It is now true.

14.4 Reply capture (live since 2026-09-23)

SES inbound is configured through `inboundEnabled: true` and `inboundDomain` in `cdk.json`. The Messaging stack owns the receipt rule set and activates it; the Compute stack sets `SES_INBOUND_DOMAIN` on the tasks and subscribes the inbound topic to the API. The only manual step was the MX record at the registrar (`InboundMxRecord` output), which is in place. Verify with:

```

aws ses describe-active-receipt-rule-set --region ap-south-1          # Metadata.Name = mailapp-
prod-inbound
aws sns list-subscriptions-by-topic --topic-arn arn:aws:sns:ap-south-1:176032258686:mailapp-
prod-ses-inbound --region ap-south-1
nslookup -type=MX reply.somvanshitechnologies.digital              # inbound-smtp.ap-south-
1.amazonaws.com

```

To move the inbound domain, change `inboundDomain`, deploy Data + Messaging + Compute, and add the new MX record. Never point the root domain's MX at SES: its mailboxes are hosted at Hostinger.

Per-user IMAP polling needs no infrastructure: each user enables it on **My profile** (Hostinger: `imap.hostinger.com`, port 993, full address, mailbox password) and tests the connection there. The organisation-wide `IMAP_*` environment variables remain available for a shared mailbox but are empty in production.

14.5 DNS records to keep in mind (registrar-managed)

Record	Status on 2026-09-23
<code>outreach.<domain></code> A/CNAME to the ALB	present (site is up)
DKIM CNAMEs for the SES identity	present (identity verified)
<code>_dmarc.<domain></code> TXT	<code>p=none</code> ; move to <code>p=quarantine</code> once reports are clean
SPF TXT	includes Hostinger only; add a custom MAIL FROM (<code>mail.<domain></code> MX to <code>feedback-smtp.ap-south-1.amazonses.com</code> , TXT <code>v=spf1 include:amazonses.com ~all</code>) and set it on the identity so SPF aligns
<code>reply.<domain></code> MX 10 <code>inbound-smtp.ap-south-1.amazonaws.com</code>	present (replies captured by SES)

15. Deployment procedure

15.1 Prerequisites

- AWS CLI authenticated as a principal allowed to deploy (the IAM user `mailer` is used today).
- Docker Desktop running (the Compute stack builds the image locally).
- `npm install` at the repo root.
- CDK bootstrapped in the account/region (already done for prod).

15.2 Code deploy (no schema change)

```
cd infra/cdk
npx cdk diff MailApp-prod-Compute
powershell -ExecutionPolicy Bypass -File deploy-watchdog.ps1 -Stacks "MailApp-prod-Compute"
```

`deploy-watchdog.ps1` runs `cdk deploy ... --require-approval never` and watches its log. From the current deployer machine the CDK asset publisher intermittently stalls forever on its first call to the ECR API: the hostname resolves to several addresses and two of them (`13.200.95.225`, `13.200.93.41` on 2026-09-23) are unreachable from that network, and the AWS SDK has no request timeout. The watchdog kills the deploy after 2 minutes without log output and retries (up to 6 times); a healthy deploy takes about 8 minutes. From a network without this problem, plain `npx cdk deploy MailApp-prod-Compute --require-approval never` is fine.

Deploy Messaging together with Compute whenever `messaging-stack.ts` changed (the events webhook subscription lives in the Messaging stack); add Data when the inbound bucket policy changes. ECS performs a rolling replacement; the circuit breaker rolls back a deployment whose tasks fail health checks. The worker runs with min healthy 0, so it is absent for one to two minutes per deploy; queued jobs simply wait.

15.3 Deploy with a new migration

1. Edit `apps/api/src/db/schema.ts`, then `npm run db:generate -w apps/api` to create `apps/api/drizzle/00NN_*.sql`. Review the SQL. Never edit an applied migration.
2. Deploy as in 15.2.
3. Run the one-off migration task using the `RunMigrationsCommand` output of the Compute stack:

```
aws cloudformation describe-stacks --stack-name MailApp-prod-Compute \
  --query "Stacks[0].Outputs[?OutputKey=='RunMigrationsCommand'].OutputValue" --output text
# then run the printed command; it is of the form:
aws ecs run-task --cluster mailapp-prod --launch-type FARGATE \
  --task-definition mailapp-prod-migrate \
  --network-configuration "awsvpcConfiguration={subnets=[...],securityGroups=
[...],assignPublicIp=ENABLED}"
```

Migrations are additive and idempotent (Drizzle journal), so running the task twice is safe. Check `/mailapp/prod/migrate` logs for the outcome (Migrations applied). Run the task only **after** the deploy has finished: the task definition family always points at the latest revision, and a run started before the deploy uses the old image and applies nothing. The API tolerates a short window where the new code runs against the old schema only if the migration is additive, which all current ones are.

15.4 First-time setup on a new environment

Follow `docs/AWS_SETUP.md`: bootstrap, set context, deploy Network and Data, put the Anthropic key into `mailapp/<env>/external`, deploy Messaging, Compute (with `subscribeWebhooks=false` until DNS points at the ALB), point DNS, redeploy Compute with `subscribeWebhooks=true`, deploy Observability, run migrations, run the seed with `SEED_ADMIN_EMAIL` / `SEED_ADMIN_PASSWORD` overrides, log in and change the password.

15.5 Rollback

- Application: redeploy the previous commit (CDK rebuilds the image) or set `imageUri` to a known good ECR image URI in `cdk.json`. ECS keeps prior task definition revisions; you can also `aws ecs update-service --task-definition <previous revision>` for an immediate rollback.
- Database: no down migrations exist. Restore from an RDS snapshot to a new instance, update the `DATABASE_URL` secret, force a new deployment.

15.6 Verifying a deploy

1. `https://outreach.somvanshitechnologies.digital/healthz` returns `{"ok":true}`; `/readyz` returns db and queue `ok`.
2. Dashboard **System** shows `sesMode=ses`, `llmProvider=anthropic`, queue counts, a recent SES sync time and the expected version.
3. Dashboard SES account card shows quota and `enforcementStatus: HEALTHY`.
4. SNS console: subscription on `mailapp-prod-ses-events` is Confirmed.
5. Run a one-lead campaign to a mailbox you own; approve; confirm arrival, the "Manage preferences" link opens the link page, Delivery and Open events appear on the lead page.
6. Reply to that email: within a minute the lead shows `replied`, and the campaign owner's mailbox receives the forwarded copy (SES path) or the reply is picked up from their inbox (IMAP path).

16. Operations runbook

`docs/OPERATIONS.md` has the detailed runbook. Highlights and additions:

16.1 Where to look

Need	Where
Is it up	<code>/healthz</code> , <code>/readyz</code> , dashboard System page, ALB target health
Logs	CloudWatch log groups <code>/mailapp/prod/api</code> , <code>/worker</code> , <code>/migrate</code> ; JSON lines with <code>reqId</code> , <code>userId</code> , <code>jobId</code> , <code>leadId</code> , <code>emailId</code>
Queue backlog / failures	System page; SQL <code>select name, state, count(*) from pgboss.job group by 1,2</code>
Why a send failed	<code>send_attempts.error</code> , <code>emails.error</code> , <code>lead last_error</code> ; log message <code>send failed</code>
Why research/draft failed	<code>lead last_error (research: ... or draft: ...)</code> ; <code>llm_calls.error</code> ; log <code>research failed</code> / <code>draft failed</code>
Webhook problems	log <code>SNS signature validation failed</code> , <code>unrecognised SES event</code> , <code>event for unknown message stored</code>
Reputation	Dashboard SES card, CloudWatch <code>AWS/SES Reputation.BounceRate</code> / <code>ComplaintRate</code> , alarms
Costs of the model	Dashboard LLM usage; <code>llm_calls</code> table
Who did what	Audit log page; <code>audit_logs</code> table

16.2 Common tasks

Task	How
Stop all sending now	Pause each active campaign, or Users page "Stop sending" per owner, or scale the worker service to 0. Setting <code>dailyCap</code> to 0 does NOT stop sending (0 = unlimited); <code>1</code> is an effective stop
Resume	Campaign Resume (re-enqueues everything pending)
Bounce or complaint spike	Pause campaigns, inspect events, check enforcement status, review lead source; addresses are already suppressed
Instructions changed	Applies to new drafts only; use Regenerate on pending drafts
Reply missed	Which Reply-To did the email carry? Inbound domain: check the SNS inbound subscription, the inbound bucket and <code>inbound_messages</code> (match method). Own mailbox: check reply polling on the user's profile (last poll, error) and the System page. Fallback: Mark replied
Contact wants back in after unsubscribing	They click Subscribe again on the link page, or an operator removes the <code>unsubscribe</code> row on the Suppressions page

Task	How
Change the link page (services shown, wording, unsubscribe button)	Services → Manage link page (admin); preview updates live
Change sender address	Must be a verified SES identity or on a verified domain; otherwise the save is rejected. Full procedure in 16.2a
Add a user	Users page; set role, optional sender identity, dashboard scope. Sender address steps in 16.2a
Grant campaign access	Campaign detail, Access section (admin)
Rotate secrets	Update Secrets Manager, <code>aws ecs update-service --force-new-deployment</code> for api and worker. Rotating <code>JWT_SECRET</code> logs everyone out; rotating <code>APP_SECRET</code> breaks unsubscribe links already sent
Scale	<code>workerDesiredCount</code> and <code>WORKER_*_CONCURRENCY</code> in CDK; respect Anthropic and SES rate limits
Backups	RDS automated 7 days; S3 storage bucket versioned
Data growth	<code>email_events</code> , <code>send_attempts</code> , <code>llm_calls</code> , <code>ses_snapshots</code> (three rows every 5 minutes) grow without bound; archive or truncate per retention policy
Start fresh (wipe all campaign data, keep configuration)	<code>infra/cdk/reset-prod-data.ps1 -Yes</code> ; details in 16.2b

16.2a Adding a sending user or a new sender address

The application never sends through a user's own mailbox. Every email leaves through SES, and the From address is whatever the campaign owner has set on **My profile** (or an admin has set on the **Users** page). When that address is saved, the API calls `ListEmailIdentities` and accepts it only if the address itself is a verified email identity, or its domain is a verified domain identity (`apps/api/src/modules/settings/sender.ts`, `assertSenderVerified`). Otherwise the save is rejected with a message that lists the verified domains. Nothing else in the app needs to change.

Because the domain identity `somvanshitechnologies.digital` is already verified, there are two very different procedures depending on the address.

Case A: address on `somvanshitechnologies.digital` (the normal case). No AWS work.

1. Create the mailbox at Hostinger if the person should receive mail there (hPanel → Emails).
2. In the dashboard, an admin opens **Users**, creates the user (name, login email, role, password) or edits an existing one, and fills **From name** and **From email**. The user can also do this later under **My profile**. Save; the SES check runs at this point.
3. Optional but recommended: on **My profile**, enable reply polling with the Hostinger IMAP settings (`imap.hostinger.com`, port 993, full address, mailbox password) and press **Test connection**. Without it, replies are still captured through the `reply.` subdomain, but only if the campaign does not override Reply-To with the person's own mailbox.
4. Send one test campaign to yourself and confirm Delivered shows on the campaign page.

Case B: address on a domain SES has never seen. Verify the domain first, then do Case A.

1. Sign in to the AWS console (account `176032258686`, region **ap-south-1**, the identity must be in this region). Open **Amazon SES** → **Configuration** → **Identities** → **Create identity**.
2. Choose **Domain**, not **Email address**. One verified domain covers every address on it; a verified email address covers that single address only and needs a click-through confirmation email for each new person. Leave the tenant option empty; the app does not use SES tenants.
3. Keep **Easy DKIM** (RSA 2048) enabled and create the identity. SES shows three CNAME records.
4. Add those three CNAMEs at the DNS provider of that domain (Hostinger: hPanel → Domains → DNS / Name Servers). Copy the Name and Value exactly; Hostinger appends the domain, so enter only the host part in Name. DKIM is what verifies the identity and what DMARC alignment relies on.
5. Add `_dmarc.<domain> TXT v=DMARC1; p=none; rua=mailto:<your address>`. This is for deliverability and reporting; it is not what makes SES mark the identity verified.
6. If that domain also hosts mailboxes elsewhere, keep its existing SPF and MX records. Do not point its MX at SES. Add a custom MAIL FROM only if SPF alignment is wanted (14.5).
7. Wait until the identity shows **Verified** and DKIM **Successful** (minutes to a few hours, depending on DNS propagation). `nslookup -type=CNAME <selector>._domainkey.<domain>` confirms the records are visible.
8. Continue with Case A. The app's identity check reads SES live, so no deploy or restart is needed.

Both procedures require the SES account to remain out of the sandbox (production access) and the enforcement status to be HEALTHY; a newly verified domain does not change the account-level quota.

16.2b Starting fresh: wiping all campaign data

Used once on 2026-09-25 after the trial period, and available for any later "clean slate". The script `infra/cdk/reset-prod-data.ps1` (run from a machine with the `mailer` CLI credentials):

```
powershell -ExecutionPolicy Bypass -File infra/cdk/reset-prod-data.ps1 -Yes
```

What it does, in order:

1. Runs `apps/api/scripts/reset-data.cjs` inside the production image as a one-off ECS task (the migrate task definition with a command override, so no deploy is needed). In one transaction it truncates `campaigns`, `leads`, `emails`, `send_attempts`, `email_events`, `llm_calls`, `inbound_messages`, `campaign_access`, `files`, `audit_logs`, `ses_snapshots`, `daily_send_counters` and `suppressions`, and deletes every queued job (`pgboss.job`, `pgboss.archive`). Cron schedules are untouched, so polling and snapshots resume by themselves.
2. Empties the upload prefix of the storage bucket and the `inbound/` prefix of the inbound mail bucket.
3. Restarts the api and worker services (`--force-new-deployment`) so the process-local counters on the System page start from zero.
4. Deletes the CloudWatch log streams of tasks that are no longer running (`-SkipLogs` keeps them).

What it keeps: users with their sender profiles, IMAP settings and sessions, organisation settings, services, instruction documents (organisation and personal), the link page configuration and the IMAP polling cursors (so old inbox mail is not re-read as replies).

The suppression list is the only data worth a copy: the task prints it as a `SUPPRESSIONS_BACKUP` line, and the script saves the whole task log under `%TEMP%\mailapp-reset`. CloudWatch metrics (`MailApp` namespace, `AWS/SES`) cannot be deleted and simply age out; the SES account-level sending statistics are likewise not affected. The same script runs locally against a development database with `npm run db:reset -w apps/api` (uses `DATABASE_URL`).

16.3 Incident: events stopped arriving

Symptom: sends succeed but Delivery/Open counts stay at zero while the SES console shows them. Check, in order: SNS subscription exists and is Confirmed; `PUBLIC_BASE_URL` matches the subscribed URL; API logs for signature failures; `SNS_ALLOWED_TOPIC_ARNS` contains the topic ARN (Compute sets it automatically). This happened on 2026-09-23 because the subscription had never been created.

16.4 Incident: SES enforcement status not HEALTHY

The worker logs an error on every sync. Pause campaigns, open the SES account dashboard, follow the review process, and reduce volume when reinstated. Bounce rate above 10 % or complaints above 0.5 % lead AWS to pause the account.

16.5 Incident: worker down

Alarm `mailapp-prod-worker-down`. Research, drafting, sending and follow-ups stall; the API keeps working. Check ECS service events and worker logs, then force a new deployment.

17. Testing and quality

Suites (see `docs/TESTING.md`):

Suite	Command	Needs
API unit (6 files)	<code>npx vitest run test/unit --root apps/api</code>	nothing
API integration (7 files)	<code>npx vitest run test/integration --root apps/api</code>	Postgres on 5433 with database <code>mailapp_test</code>
Dashboard	<code>npm run test -w apps/web</code>	nothing (jsdom)
CDK synth assertions	<code>npm run test -w infra/cdk</code>	nothing
Type check	<code>npm run typecheck</code>	nothing
Build	<code>npm run build</code>	nothing

Create the integration database once:

```
docker compose up -d postgres
docker exec mailapp-postgres psql -U mailapp -d mailapp -c "CREATE DATABASE mailapp_test"
```

Integration tests run migrations, truncate tables before each test, and drive the whole pipeline in-process with `MemoryQueue.drain()`, the mock LLM and the mock SES gateway. They cover auth and RBAC, campaign import and preview, research to send, auto mode and validator retry, pause/resume, regenerate and approve-all, SES event ingestion (all types, dedupe, topic allow-list, subscription confirmation), inbound replies, follow-ups and threading, send window, daily cap, suppression at send time, permanent SES errors, send-test, settings, catalogue, the link page (services, buttons, unsubscribe confirmation, subscribe again, preview), analytics, system status, the access and sender-profile rules, reply capture (Reply-To resolution, forwarding, encrypted IMAP credentials, polling with a fake IMAP server, cursor handling, per-mailbox errors).

Baseline on 2026-09-24: API 86/86, dashboard 11/11, CDK 16/16, typecheck and build clean. There is no CI pipeline; add one (typecheck, unit, web, cdk on every push; integration with a Postgres service).

18. Known issues, gaps and technical debt

Ordered by impact.

#	Item	Impact	Suggested action
1	<code>alertEmail</code> empty	Alarms unseen	Set it and redeploy Observability
2	Deploys from the current machine stall on unreachable ECR addresses	Deploys take 8 to 45 minutes	Use <code>deploy-watchdog.ps1</code> (15.2) or deploy from another network
3	<code>docs/API.md</code> lacks the 2026-09-23/24 endpoints	Integrators misled	Merge section 12 into it
4	DMARC <code>p=none</code> , no custom MAIL FROM	Weaker deliverability, no SPF alignment	Section 14.5 (optional by decision)
5	No git remote (by decision) and no CI pipeline	Single copy of the code; regressions reach prod	Keep backups of the working directory; add CI if a remote is adopted later
6	Single worker task with in-process token bucket	Adding workers multiplies the send rate	Keep <code>WORKER_SEND_CONCURRENCY</code> x <code>tasks</code> under the SES rate, or move the bucket to Postgres
7	Single-AZ RDS, 1 API task, no autoscaling, no NAT	Acceptable for an internal tool; downtime on AZ or task failure	Document the decision; raise counts if usage grows
8	<code>dailyCap = 0</code> means unlimited	Operators may think 0 pauses	UI hint exists in the runbook; consider a separate "sending paused" switch
9	Settings cache of 10 s and per-process	Setting changes take up to 10 s to reach the worker	Acceptable
10	<code>ses_snapshots</code> and <code>email_events</code> grow forever	Storage and slower queries	Retention job
11	Header alias <code>name</code> maps to <code>first_name</code>	Full-name columns become first names	Rename the column before upload or split it
12	Send window search is hourly, minutes are truncated	A send scheduled at 08:00 goes at the next hour boundary	Fine for current use
13	<code>campaign.export</code> queue defined but unused	Dead code	Remove or implement async export for big sheets
14	SES <code>Subscription</code> events unsubscribe the lead	Correct for SES subscription management, but no test of the live path	Verify once with a real one-click unsubscribe
15	<code>PLAN.md</code> still lists the original phase plan and some metrics names that changed	Minor confusion	Update or mark historical

#	Item	Impact	Suggested action
	(llm_tokens is now llm_input_tokens / llm_output_tokens)		
16	apps/web/dist served only if present in the image	Missing build makes / 404	The Dockerfile always builds it; keep it that way
17	Prompt caching depends on the system blocks being identical	Changing services or instructions mid-campaign invalidates the cache and raises cost	Batch instruction edits
18	Unsubscribe button hidden by default on the link page	Recipients can only opt out by replying or via their mail client's one-click unsubscribe	Intentional (known contacts only); switch it on under Services → Manage link page if the audience changes
19	Text-only sends (personal mode with tracking off) show the full link-page URL	Cosmetic	Keep trackOpens on, or accept the URL in the plain-text part
20	IMAP polling lookback is 3 days on first contact	Replies older than that when polling is enabled are never matched	Enable polling before sending, or use Mark replied

19. Accounts, credentials and third parties

No secret values are written in this document. The new owner needs access to each of these.

Item	Where / who	Notes
AWS account <code>176032258686</code>	Root or IAM admin	Needed for CDK deploys, Secrets Manager, SES console, RDS snapshots
IAM user <code>mailer</code>	Local AWS CLI profile of the previous owner	Used for <code>cdk deploy</code> ; create your own principal rather than sharing keys
Secrets Manager <code>mailapp/prod/*</code>	AWS	DB URL, JWT and app secrets, Anthropic key, IMAP password
Anthropic API key	Anthropic console (organisation account)	Stored only in <code>mailapp/prod/external</code> ; local <code>.env</code> may hold a second key
Domain <code>somvanshitechnologies.digital</code>	Registrar (Hostinger)	DNS for the app hostname, DKIM, DMARC, SPF, future MX
ACM certificate	AWS ap-south-1	Renewal is automatic only if the validation CNAME stays in DNS
SES production access	AWS SES console	Confirm it remains granted
Dashboard admin login	Application	Seeded admin; rotate the password on handover, create named admin users, deactivate the previous owner's account
Local <code>.env</code>	Previous owner's machine	Contains local secrets; transfer securely or regenerate
Docker Desktop	Deployer machine	Required for image builds by CDK

Third-party dependencies with cost implications: Anthropic API (per-token, research uses web search and can take many turns per lead), Amazon SES (per message), RDS (t4g.micro), Fargate (two tasks 0.5 vCPU each), CloudWatch (custom metrics in namespace `MailApp`).

20. Handover checklist

Tick each item during the transition.

Repository and code

- ☒ Storage module committed and `.gitignore` anchored (2026-09-23)
- ☐ Hand over the working directory including `.git` (no remote by decision)
- ☐ On the new machine, `npm install`, `npm run typecheck`, `npm run build` succeed
- ☒ All test suites run and baseline recorded (section 17, 2026-09-24)
- ☐ Add a CI pipeline if a hosted remote is adopted

Access

- ☐ New AWS IAM principal(s) created for deploys; previous keys rotated or deleted
- ☐ Access to Secrets Manager, SES console, RDS, CloudWatch confirmed
- ☐ Registrar / DNS access confirmed
- ☐ Anthropic console access confirmed; key ownership transferred or rotated
- ☐ Dashboard: new admin users created, seeded/previous admin password rotated

Production hygiene

- ☐ `alertEmail` set; alarm email subscription confirmed
- ☒ Reply capture enabled (SES inbound domain + IMAP) and deployed (2026-09-23)
- ☐ Reply capture tested end to end with a real reply on the new owner's machine
- ☐ DMARC/SPF/custom MAIL FROM records improved (14.5, optional)
- ☒ SNS subscriptions Confirmed (events and inbound); Delivery and Open events flow (2026-09-23)
- ☐ Each sending user has set their sender identity on My profile (and reply polling if they use their own Reply-To); new users follow 16.2a
- ☐ Link page reviewed under Services → Manage link page (contact email set, service URLs filled)
- ☐ SES enforcement status HEALTHY and production access active
- ☐ RDS snapshot taken at handover

Knowledge

- ☐ Walk through one campaign end to end with the previous owner (upload, review, send, events, export)
- ☐ Review the product decisions in section 1 with the business owner
- ☐ Agree a data retention policy (section 16.2)

21. Appendix A: environment variables

From `apps/api/src/config.ts` and `.env.example`. Defaults apply when unset.

Variable	Default	Purpose
<code>NODE_ENV</code>	development	development, test, production
<code>PORT</code>	4000	API port
<code>LOG_LEVEL</code>	info	pino level
<code>PUBLIC_BASE_URL</code>	http://localhost:4000	Used in unsubscribe links and SNS subscription
<code>WEB_ORIGIN</code>	http://localhost:5173	CORS origin
<code>DATABASE_URL</code>	required	Postgres connection string
<code>JWT_SECRET</code>	required, 32+ chars	Access token signing
<code>APP_SECRET</code>	required, 32+ chars	Cookie signing and unsubscribe HMAC
<code>ACCESS_TOKEN_TTL_MINUTES</code>	15	
<code>REFRESH_TOKEN_TTL_DAYS</code>	14	
<code>SEED_ADMIN_EMAIL</code> , <code>SEED_ADMIN_PASSWORD</code>	unset	Used by the seed when no users exist (password 12+ chars)
<code>LLM_PROVIDER</code>	mock	anthropic or mock
<code>ANTHROPIC_API_KEY</code>	empty	Required in production with anthropic
<code>LLM_MODEL</code> , <code>LLM_RESEARCH_MODEL</code>	claude-opus-5	Env defaults for the DB settings
<code>LLM_MAX_CONCURRENCY</code>	3	In-flight model calls per process
<code>LLM_WEB_SEARCH</code>	true	Enables server-side web search in research
<code>WEBSITE_FETCH_ENABLED</code>	true	Fetch the lead website before research
<code>AWS_REGION</code>	us-east-1	SES, S3, CloudWatch region (ap-south-1 in prod)
<code>AWS_ACCESS_KEY_ID</code> , <code>AWS_SECRET_ACCESS_KEY</code>	empty	Leave empty to use the default credential chain / task role
<code>SES_MODE</code>	mock	ses or mock
<code>SES_CONFIGURATION_SET</code>	empty	Configuration set name
<code>SES_FROM_EMAIL</code> , <code>SES_FROM_NAME</code> , <code>SES_REPLY_TO</code>	example values	Env defaults for settings
<code>SES_DAILY_CAP</code>	2000	Env default; 0 = unlimited
<code>SES_MAX_SEND_RATE</code>	5	Per second

Variable	Default	Purpose
SES_INBOUND_BUCKET , SES_INBOUND_PREFIX	empty, inbound/	SES receipt rule storage
SES_INBOUND_DOMAIN	empty	Domain whose MX points at SES; generated Reply-To addresses use it (reply.somvanshitechnologies.digital in prod)
SNS_ALLOWED_TOPIC_ARNS	empty	Comma-separated allow-list; empty accepts any verified SNS message
SNS_VERIFY_SIGNATURES	true	Only false in tests; forced true in production
IMAP_ENABLED , IMAP_HOST , IMAP_PORT , IMAP_USER , IMAP_PASSWORD , IMAP_MAILBOX	false, empty, 993, empty, empty, INBOX	Optional organisation-wide mailbox to poll; users' own mailboxes are configured in the app
STORAGE_DRIVER	local	local or s3
STORAGE_LOCAL_DIR	./storage	Local uploads folder (/data/storage in the image)
STORAGE_S3_BUCKET , STORAGE_S3_PREFIX	empty, mailapp/	S3 storage
CLOUDWATCH_METRICS_ENABLED	false	Push custom metrics
CLOUDWATCH_NAMESPACE	MailApp	
CLOUDWATCH_SES_METRICS_ENABLED	false	Pull AWS/SES metrics into the dashboard
WORKER_RESEARCH_CONCURRENCY , WORKER_DRAFT_CONCURRENCY , WORKER_SEND_CONCURRENCY	2, 2, 2	Worker pollers per queue
APP_VERSION	1.0.0	Shown on the System page
LOG_PRETTY	unset	Set false in the image; pretty logs in dev

22. Appendix B: converting this document to PDF

The file is plain GitHub-flavoured Markdown with HTML page-break markers (`<div style="page-break-after: always;"></div>`) that every HTML-based converter honours. Tables are kept to four columns so they fit portrait A4. Diagrams are plain text in code blocks so they render everywhere.

Option 1, VS Code: install the "Markdown PDF" extension (yzane), open `docs/HANDOVER.md`, run **Markdown PDF: Export (pdf)**. Page breaks and tables render as-is.

Option 2, Node (no global install):

```
npx md-to-pdf docs/HANDOVER.md
# produces docs/HANDOVER.pdf using a headless Chromium
```

Option 3, headless Chrome or Edge already on the machine:

```
npx -y marked -i docs/HANDOVER.md -o docs/HANDOVER.html --gfm
# open docs/HANDOVER.html in the browser and print to PDF, or:
"C:\Program Files\Google\Chrome\Application\chrome.exe" --headless --disable-gpu \
  --print-to-pdf="docs\HANDOVER.pdf" --no-pdf-header-footer "file:///<repo
path>/docs/HANDOVER.html"
```

Option 4, pandoc (if installed): `pandoc docs/HANDOVER.md -o docs/HANDOVER.pdf --pdf-engine=xelatex -V geometry:margin=2cm`. Pandoc's LaTeX route ignores the HTML page-break markers; add `--from gfm` and accept continuous flow, or use the HTML route above.

Keep this document next to the code and update the "Document version", "Date" and "Code baseline" fields whenever it changes.